

Exp:6 A* Search Algorithm

Program:

```
import heapq

def a_star_search(graph, heuristic, start, goal):
    priority_queue = [(heuristic[start], 0, start, [start])]
    visited = set()

    while priority_queue:
        f, g, node, path = heapq.heappop(priority_queue)

        if node in visited:
            continue

        visited.add(node)

        if node == goal:
            return path, g

        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                g_new = g + cost
                f_new = g_new + heuristic[neighbour]
                heapq.heappush(priority_queue,
                               (f_new, g_new, neighbour, path + [neighbour]))

    return None, float("inf")

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 6)],
    'E': [('G', 1)],
    'F': [('G', 4)],
    'G': []
}
```

```

heuristic = {

    'A': 6,

    'B': 4,

    'C': 5,

    'D': 3,

    'E': 2,

    'F': 4,

    'G': 0

}

path, cost = a_star_search(graph, heuristic, 'A', 'G')

print("Optimal Path :", " -> ".join(path))

print("Total Cost :", cost)

```

Output:

The image shows a code editor window on the left and an IDLE Shell window on the right. The code editor contains a Python script for an A* search algorithm. The script defines a graph, a heuristic function, and the A* search function. The graph has nodes A through G with their respective heuristic values and edges with costs. The A* search function finds the optimal path from node A to node G. The output of the script is displayed in the IDLE Shell window.

```

import heapq
# A* Search Algorithm
def a_star_search(graph, heuristic, start, goal):
    priority_queue = [(heuristic[start], 0, start, [start])] # (f, g, node, path)
    visited = set()
    while priority_queue:
        f, g, node, path = heapq.heappop(priority_queue)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            return path, g
        for neighbour, cost in graph[node]:
            if neighbour not in visited:
                g_new = g + cost
                f_new = g_new + heuristic[neighbour]
                heapq.heappush(priority_queue, (f_new, g_new, neighbour, path + [neighbour]))
    return None, float("inf")

# Weighted Graph
graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 6)],
    'E': [('G', 1)],
    'F': [('G', 4)],
    'G': []
}

# Heuristic Values
heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 3,
    'E': 2,
    'F': 4,
    'G': 0
}

path, cost = a_star_search(graph, heuristic, 'A', 'G')
print("Optimal Path :", " -> ".join(path))
print("Total Cost :", cost)

```

The IDLE Shell window shows the output of the script:

```

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/Sakshi/OneDrive/Documents/MLA01 - Lab Exp/Exp-6 A Search algorithm/E
xp-6 A(star) Search algorithm program
Optimal Path : A -> B -> E -> G
Total Cost : 8
>>>

```